Hybrid LSTM-Transformer for RONIN Heading Estimation

Model

Model Name	Hybrid LSTM-Transformer
Name	Gopal Datt Bhatt

4 Section 1 — Architecture

4.1 1.1 Model Description

This model takes a window of inertial and magnetic sensor data with shape (200, 9). A stacked LSTM first extracts temporal motion features from the sequence, then a Transformer encoder processes the LSTM feature sequence to capture longer-range dependencies across the full one-second window. The final fused representation is passed through fully connected layers to predict sin(heading) and cos(heading).

4.2 1.2 Layer Structure

Component	Details
Input	(200 x 9)
LSTM Section	2-layer LSTM, hidden size 64, dropout 0.3
Transition Point	Full LSTM output sequence (batch, 200, 64) is linearly projected to transformer dimension 128
Transformer Section	Positional encoding used; 2 encoder layers; 2 attention heads; model dimension 128; feedforward dimension 128; dropout 0.2; norm_first=True
How outputs combined	Mean pooling is applied to the transformer output and concatenated with the final LSTM hidden state before the fully connected block
Fully Connected Output	192 -> 64 -> 2 output neurons [**sensor window → LSTM/Transformer features → fused 192 features → dense layer → 64 features → final 2 heading values**]

4.3 1.3 Trainable Parameters

Total trainable parameters: 272,450.

4.4 1.4 Architectural Choices and Justification

A hybrid design was chosen to combine two useful properties. The LSTM front-end is good at extracting local temporal motion patterns from IMU and magnetometer sequences, while the Transformer encoder can model broader context across the full window using self-attention and positional encoding. This follows the common sequence-modelling rationale that recurrent layers capture short-range sequential structure and Transformer layers add longer-range contextual reasoning.

The LSTM hidden size was kept at 64 to keep the model computationally manageable on CPU while still giving the recurrent block enough capacity to learn motion dynamics. The transformer dimension was set to 128 so that the projected sequence has a richer representation than the raw LSTM state. Two

attention heads were used because they divide the 128-dimensional transformer space cleanly and keep the model lightweight. Two encoder layers were used to add depth without making training prohibitively slow. The 192 -> 64 -> 2 output head is a compact fusion block that combines the recurrent and attention-based summaries before predicting the circular heading target.

5 Section 2 — Training Setup

5.1 2.1 Loss Function

The loss function used was mean squared error (MSE) on the two output values [sin(heading), cos(heading)]. This is appropriate because heading is a circular quantity, and predicting sine and cosine avoids the discontinuity between 0 degrees and 360 degrees. Two angles that are close to each other remain close in the sine-cosine space even when they cross the wraparound boundary.

5.2 2.2 Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	256
Number of epochs	15

The model was trained using the Adam optimizer with a learning rate of 0.001. A batch size of 256 was used to reduce the number of parameter updates per epoch and improve training efficiency on CPU. The maximum number of training epochs was set to 15, with early stopping applied based on validation loss to avoid unnecessary training and reduce overfitting.

5.3 2.3 Learning Rate Scheduling

Learning rate scheduling was used through ReduceLROnPlateau based on validation loss. The scheduler used a factor of 0.5, patience of 3 epochs, and a minimum learning rate of 1e-6. The learning rate remained at 0.001 during the first 11 epochs and was reduced to 0.0005 from epoch 12 onward after validation improvement slowed. This helped the model continue training with smaller and more stable parameter updates.

5.4 2.4 Early Stopping

Yes. Early stopping was used with patience 7 and the monitored metric was validation loss. In the executed run, training printed “Early stopping triggered” at the end of epoch 15, which was also the maximum epoch limit for this notebook.

5.5 2.5 Training Difficulties

The main difficulty was training time. The notebook was run on CPU, and each epoch was slow because the model combines a recurrent encoder with a Transformer encoder on a large number of overlapping windows. Overfitting also became a concern: the training loss kept decreasing, but the validation loss improved only up to a point and then flattened or worsened slightly. Dropout, learning-rate reduction on plateau, and early stopping were used to control this.

6 Section 3 — Results

6.1 3.1 Quantitative Results

Metric	Value
MAE on test set (degrees)	66.3092

RMSE on test set (degrees)	82.9964
Approximate training time	19,949.23 seconds (~5.54 hours)

6.2 3.2 Predicted vs Ground Truth Trajectory

The figure below shows model predictions against ground truth for one full unseen test trajectory, sequence a006_2. The plot covers the full sequence and contains 3,475 window-level predictions across the trajectory.

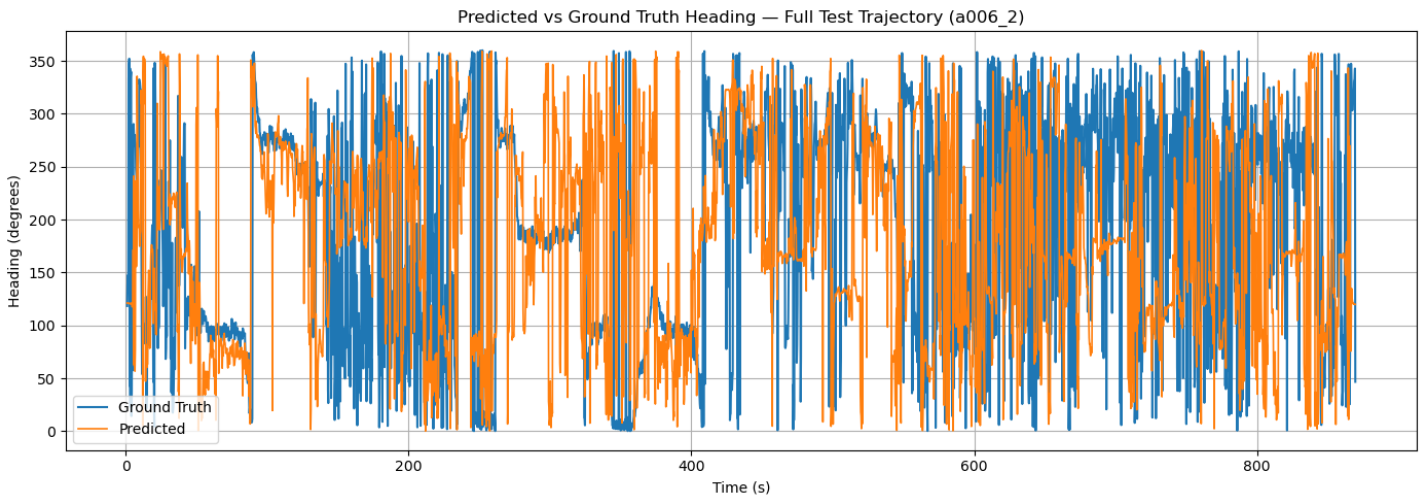

Figure 1. Predicted vs ground truth heading over one full test trajectory (a006_2).

6.3 3.3 Loss Curve

The training and validation loss curves from the executed notebook are shown below.

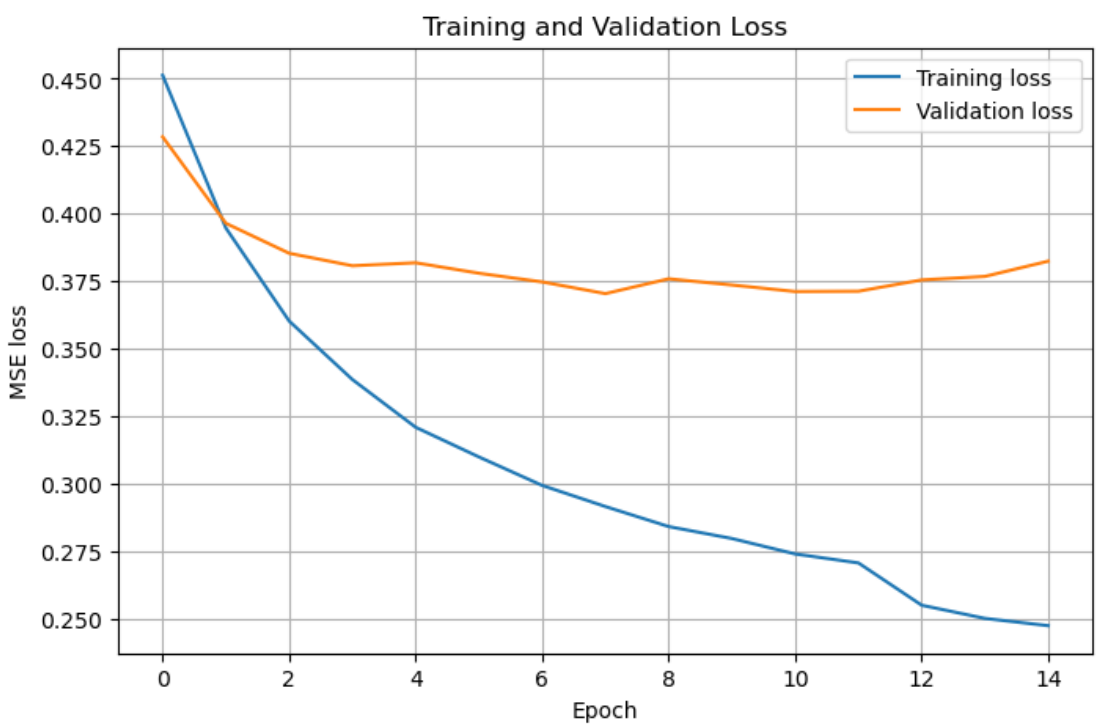

Figure 2. Training and validation MSE loss over 15 epochs.

6.4 3.4 MAE Breakdown by Scenario

Scenario	MAE (degrees)
Straight walking	59.2561
Sharp turns	89.6632
Short trajectories	65.0571
Long trajectories	67.1442

Scenario-wise MAE was computed using circular heading error on the unseen test set. Straight walking was defined as windows with small ground-truth heading change, while sharp turns were defined as windows with large ground-truth heading change. Short and long trajectories were separated using a median split based on trajectory length in windows. The median trajectory length was 2551.5 windows, so trajectories with 2551 windows or fewer were treated as short and trajectories with 2552 windows or more were treated as long

7 Section 4 — Observations and Technical Reflection

7.1 4.1 Where the Model Performs Well

The model performs relatively better in portions of the trajectory where heading changes are smoother and the target stays within a narrower angular range for a short period. In those segments, the recurrent encoder can exploit local temporal consistency and the Transformer can reinforce stable contextual patterns across nearby windows. The loss curve also shows that the model learned meaningful structure during training because both training and validation losses dropped substantially in the early epochs.

7.2 4.2 Where the Model Struggles

The model struggles on difficult parts of the full unseen test trajectory where heading changes rapidly, wraps around the circular boundary, or appears noisy and unstable. This is consistent with the relatively high test MAE of 66.3092 degrees and RMSE of 82.9964 degrees. In the full-trajectory plot, there are many sections where the predicted curve does not align closely with the ground truth, especially during abrupt changes and highly variable regions.

7.3 4.3 Loss Curve Interpretation

The loss curve shows clear learning at the start of training. Training loss decreases monotonically from about 0.451 to 0.247. Validation loss also improves early, falling from about 0.428 to its best region around 0.370 to 0.371 by roughly epochs 8 to 11. After that, training loss continues to fall while validation loss stops improving and drifts upward. This suggests moderate overfitting rather than underfitting. The model has enough capacity to fit the training windows, but its generalisation to unseen subjects remains limited.

7.4 4.4 Architecture vs Training — Root Cause of Limitations

The observed limitations likely come from both the architecture and the training conditions. The architecture is expressive enough to learn the training set, but the final test errors show that it is still not robust on unseen trajectories. Part of the problem may be architectural: a simple mean-pooled Transformer summary and compact fully connected head may not capture all heading dynamics. Part of the problem is also training-related: the model was trained on CPU with a lightweight configuration chosen for runtime reasons, and no scenario-specific tuning or additional feature engineering was applied. The evidence from the loss curve suggests that training data fit improved faster than validation performance, so generalisation is the key limitation.

7.5 4.5 One Change I Would Make

If more time were available, the single most useful change would be to improve the evaluation-ready model capacity while training on GPU. A stronger configuration could increase representation quality without the severe CPU time penalty that constrained this notebook. With GPU training, it would also be practical to test more runs, tune the hidden dimensions and attention settings, and possibly add a better sequence-level fusion strategy for the final prediction.